SkillMatrix

Historie użytkownika, przypadki użycia i diagramy klas

Jan Kaszycki Aleksandr Kuraskin

Inżynieria Oprogramowania 1 — 2026

Problem i pomysł

Problem: kandydat nie wie, jak jego umiejętności mają się do wymagań rynku. CV opisuje je ogólnikowo, oferty — każda po swojemu, a porównanie robi się ręcznie.

SkillMatrix:

- umiejętności jako graf atomowych umiejętności, każda z poziomem 0–1,
- poziom szacuje AI na podstawie CV, ocen i kodu z GitHuba,
- oferty pobieramy automatycznie z portali, a AI zamienia je na ten sam graf,
- system wskazuje dopasowane oferty, braki i czego się nauczyć,
- ocena i generowanie CV.

Poziomy pokazujemy opisowo (Początkujący . . . Ekspert). Jeśli kandydat bardzo chce, może włączyć liczby i poprawić ocenę AI ręcznie — ale tego nie zalecamy.

Kto i co działa w systemie

Aktor	Rodzaj	Co robi
Kandydat	człowiek	Wgrywa CV, oceny, link do GitHuba; ogląda swój graf, oferty, rekomendacje, CV. Jedyny człowiek w systemie.
Serwer SkillMatrix	nasz program	Trzyma konta, profile, oferty i graf; liczy dopasowanie i rekomendacje.
Agent ofert	nasz program	Robot: według harmonogramu pobiera ogłoszenia z portali z pracą i oddaje je serwerowi. Nie używa AI.
Agent AI	nasz program	„Mózg": ocenia umiejętności kandydata, zamienia treść ofert na umiejętności z grafu, ocenia CV. Pełni rolę administratora grafu — dodaje węzły, scala duplikaty.
Model językowy (LLM)	zewnętrzny	Np. Claude API: dostaje tekst i instrukcję, zwraca JSON. Używa go tylko Agent AI.
GitHub API, portale	zewnętrzne	Źródła danych o kodzie kandydata i ofert pracy.

Nie ma rekruterów ani administratora-człowieka.

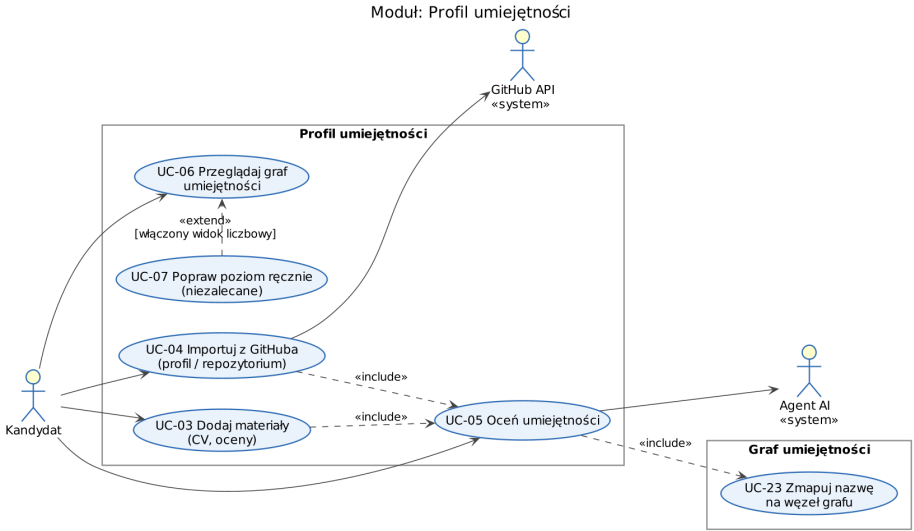
Historie użytkownika — Kandydat

ID	Jako *kandydat* chcę . . . , aby . . .	Prio
US-02/03	wgrać CV i oceny, aby system poznał moje doświadczenie bez przepisywania	M
US-04	podać link do profilu lub repozytorium GitHub, aby ocena opierała się na faktycznym kodzie	M
US-05	żeby AI oszacowało moje umiejętności 0–1, aby dostać mierzalny obraz tego, co umiem	M
US-06	widzieć graf z poziomem opisowym i uzasadnieniem, aby rozumieć ocenę bez fiksowania się na liczbach	M
US-07	móc (niezalecane) włączyć liczby i poprawić ocenę AI, aby skorygować ocenę, z którą się nie zgadzam	C
US-08	widzieć oferty posortowane wg dopasowania, aby aplikować tam, gdzie mam szanse	M
US-09	widzieć braki dla wybranej oferty, aby wiedzieć, co dzieli mnie od tej pracy	M
US-11	dostać umiejętności, które odblokują najwięcej ofert, aby uczyć się tego, co się opłaca	M
US-13/14	ocenić i wygenerować CV, aby lepiej pokazać, co umiem	S

Historie użytkownika — Agent AI i Agent ofert

ID	Historia	Prio
US-20	Jako *agent AI* chcę zamieniać treść ogłoszenia na wymagania z grafu (poziom, obowiązkowe?), aby oferty dało się porównać z profilami	M
US-21	Jako *agent AI* chcę przed dodaniem umiejętności sprawdzić, czy nie istnieje pod inną nazwą, aby w grafie nie było duplikatów	M
US-22	Jako *agent AI* chcę dodawać nowe umiejętności jako tymczasowe, aby jednorazowe pomyłki nie zaśmiecały grafu	S
US-23	Jako *agent AI* chcę okresowo scalać duplikaty, aby naprawić te, które przeszły przy dodawaniu	S
US-24	Jako *agent AI* chcę wyznaczać relacje ($\text{C++} \geq 0.8 \Rightarrow \text{C} \geq 0.6$), aby wnioskować o umiejętnościach pokrewnych	S
US-30	Jako *agent ofert* chcę cyklicznie pobierać ogłoszenia z portali, aby baza ofert była aktualna bez udziału człowieka	M
US-31	Jako *agent ofert* chcę rozpoznawać tę samą ofertę z różnych portali, aby kandydat nie widział jej kilka razy	S
US-32	Jako *agent ofert* chcę oznaczać zniknięte oferty jako nieaktualne, aby kandydat nie widział wygasłych	S

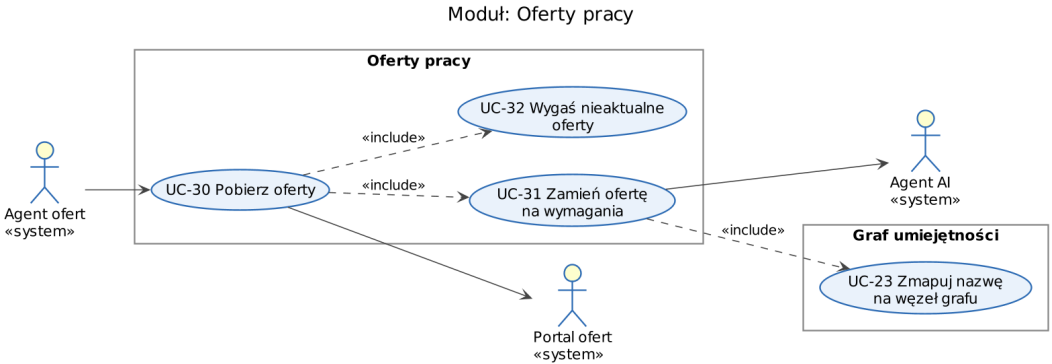
Przypadki użycia — Profil umiejętności



UC-05 Oceń umiejętności

Aktorzy	Kandydat (główny), Agent AI
Wyzwalacz	Dodanie materiału (UC-03, UC-04) lub „oceń ponownie”
Warunek wstępny	Kandydat zalogowany, ma co najmniej jeden materiał
Warunek końcowy	Profil ma poziom, pewność i uzasadnienie dla każdej umiejętności; zdarzenie `ProfileUpdated`
Scenariusz główny	1. System zbiera materiały kandydata (CV, oceny, GitHub). 2. Wysyła je do Agenta AI. 3. Agent AI zwraca oszacowania (nazwa, poziom 0–1, pewność, uzasadnienie). 4. Każda nazwa jest mapowana na węzeł grafu (UC-23). 5. System propaguje poziomy po relacjach `IMPLIES`. 6. Zapisuje ocenę i aktualizuje profil. 7. Pokazuje graf z poziomami opisowymi.
Alternatywy	6a. Kandydat ręcznie poprawił poziom → nowa wartość AI zapisana obok, korekta nie jest nadpisywana. 7a. Włączony widok liczbowy → dodatkowo wartości 0–1.
Błędy	Agent AI / LLM niedostępny, timeout 60 s; niepoprawny format odpowiedzi
Stan po błędzie	Profil bez zmian, materiały zapisane, komunikat „spróbuj później”

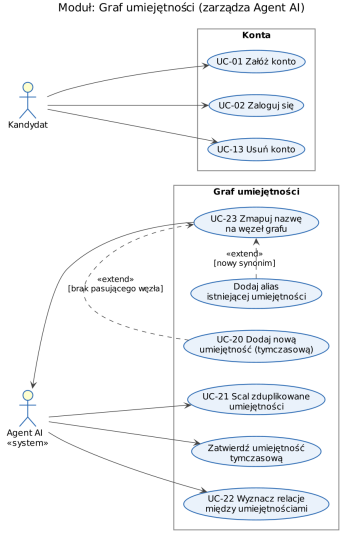
Przypadki użycia — Oferty pracy



UC-30 Pobierz oferty

Aktorzy	Agent ofert (główny), Portal ofert, Agent AI
Wyzwalacz	Termin z harmonogramu źródła (plik konfiguracyjny)
Warunek wstępny	Źródło ofert skonfigurowane i włączone
Warunek końcowy	Oferty dodane / zaktualizowane / wygaszone; zapisany `IngestionRun`; zdarzenie `OffersUpdated`
Scenariusz główny	1. Agent ofert pobiera listę włączonych źródeł. 2. Pobiera ogłoszenia od ostatniego przebiegu (adapter API / scraper). 3. Odrzuca duplikaty (`externalId`, odcisk treści). 4. Agent AI zamienia każdą nową ofertę na wymagania z grafu (UC-31, UC-23). 5. Agent ofert przesyła paczkę do serwera (`upsertOffers`). 6. Serwer wygasza oferty, których już nie ma (UC-32). 7. Agent zapisuje raport przebiegu.
Alternatywy	3a. Zmieniona treść istniejącej oferty → ponowne wyodrębnienie wymagań. 4a. Brak rozpoznanych wymagań → oferta zapisana, ale pominięta w dopasowaniu.
Błędy	Portal niedostępny lub zmienił strukturę; Agent AI niedostępny
Stan po błędzie	Pozostałe źródła działają dalej; błąd w logu; oferty z tego źródła nie są wygaszane

Przypadki użycia — Graf umiejętności (zarządza Agent AI)



Problem: AI może tworzyć niejednoznaczne węzły

Co może pójść źle

- synonimy: *JS* / *JavaScript*
- pisownia, wersje: *ReactJS* / *React 18*
- język: *bazy danych* / *databases*
- szczegółowość: *React* / *React Hooks*
- ta sama oferta → różne nazwy
- „praca w zespole" jako węzeł

Skutek: kandydat z *JavaScript* nie pasuje do oferty z *JS*.

Jak zapobiegamy

1. AI najpierw wybiera z istniejących węzłów, nowy to ostateczność (UC-23)
2. normalizacja nazw + aliasy zamiast nowych węzłów
3. podobieństwo znaczeniowe (embeddingi): ≥ 0.92 alias, 0.75–0.92 decyduje LLM wybierając z listy, < 0.75 nowy węzeł
4. graf startowy z gotowej klasyfikacji (np. ESCO)
5. temperatura 0, stałe prompty, odpowiedź wg schematu JSON
6. nowy węzeł tymczasowy do czasu wystąpienia w ≥ 3 ofertach
7. nocne scalanie duplikatów + dziennik zmian z możliwością cofnięcia

UC-23 Zmapuj nazwę na węzeł grafu

Aktorzy	Agent AI
Wyzwalacz	Krok UC-05 lub UC-31 — pojawia się nazwa umiejętności do zapisania
Warunek wstępny	Graf zawiera węzły z aliasami i wektorami podobieństwa
Warunek końcowy	Nazwa wskazuje dokładnie jeden węzeł; ewentualna zmiana grafu zapisana w dzienniku z uzasadnieniem
Scenariusz główny	1. Agent normalizuje nazwę (małe litery, bez wersji, kropek, myślników). 2. Szuka węzła o tej nazwie lub aliasie — znaleziony: koniec. 3. Liczy embedding nazwy z kontekstem i pobiera 5 najbardziej podobnych węzłów. 4. Podobieństwo najlepszego ≥ 0.92: dodaje nazwę jako alias i zwraca ten węzeł.
Alternatywy	4a. Podobieństwo 0.75–0.92: LLM wybiera węzeł z listy kandydatów albo „nowa" $\rightarrow$ alias. 4b. < 0.75 lub „nowa": tworzony węzeł `PROVISIONAL` (UC-20). 4c. LLM uznaje nazwę za nie-umiejętność $\rightarrow$ odrzucona.
Błędy	LLM niedostępny w kroku 4a
Stan po błędzie	Nazwa w kolejce do ponownego przetworzenia; graf bez zmian

Przypadki użycia — Dopasowanie, rekomendacje, CV

Moduły: Dopasowanie i rekomendacje, CV

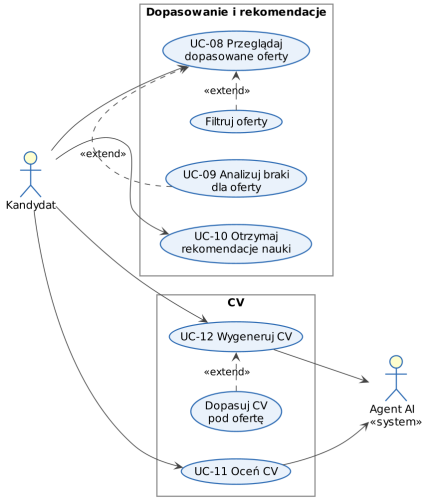
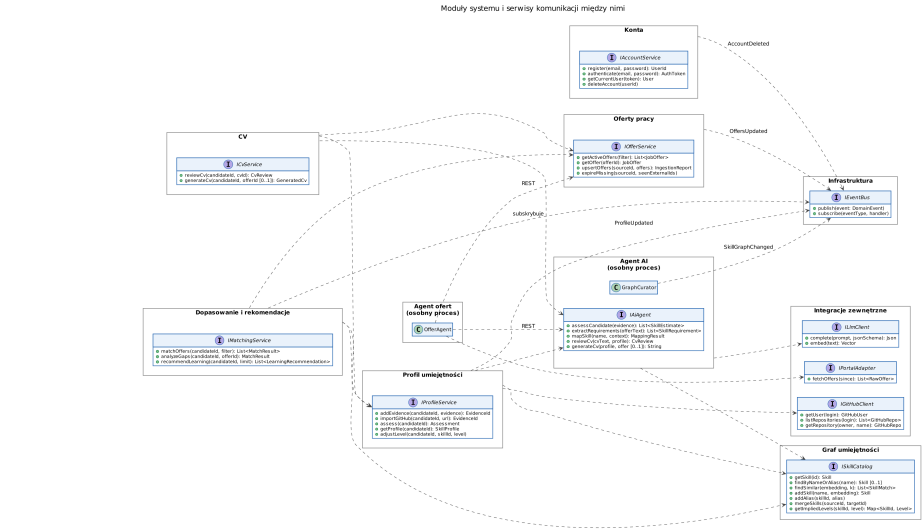


Diagram klas — moduły i serwisy komunikacji



Komunikacja między modułami

- Moduły nie sięgają do cudzych danych — tylko przez interfejsy serwisów: `IProfileService`, `IOfferService`, `ISkillCatalog`, `IMatchingService`, `ICvService`, `IAiAgent`.
- Zmiany są ogłaszane zdarzeniami na `IEventBus`: `ProfileUpdated`, `OffersUpdated`, `SkillGraphChanged`, `AccountDeleted` — np. dopasowanie unieważnia wtedy swoją pamięć podręczną.
- Agent ofert i Agent AI to osobne procesy — rozmawiają z serwerem przez REST.
- Tylko Agent AI rozmawia z modelem językowym (`ILlmClient`) — zmiana dostawcy LLM nie dotyka reszty systemu.
- Każdy portal ma swój adapter (`IPortalAdapter`) — dodanie portalu to nowa klasa.
- Graf umiejętności jest wspólnym językiem: tymi samymi węzłami opisany jest profil i wymagania ofert.

Diagram klas — graf i profil umiejętności

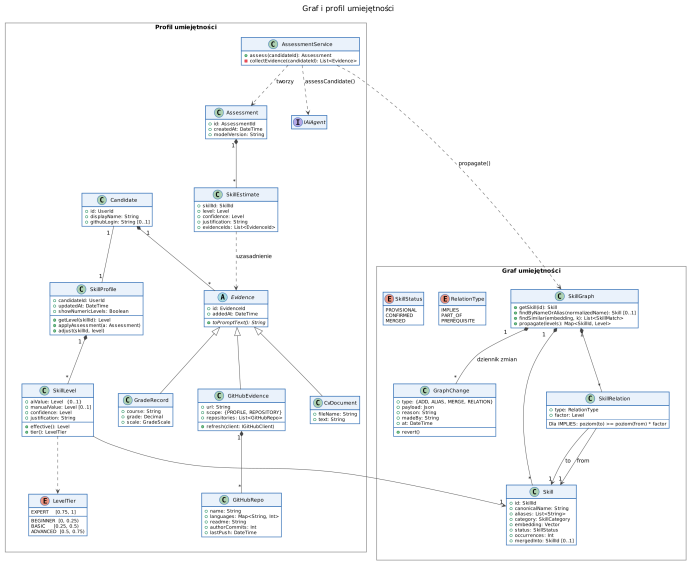


Diagram klas — Agent AI

Agent AI — ocena, przetwarzanie ofert i dbanie o spójność grafu

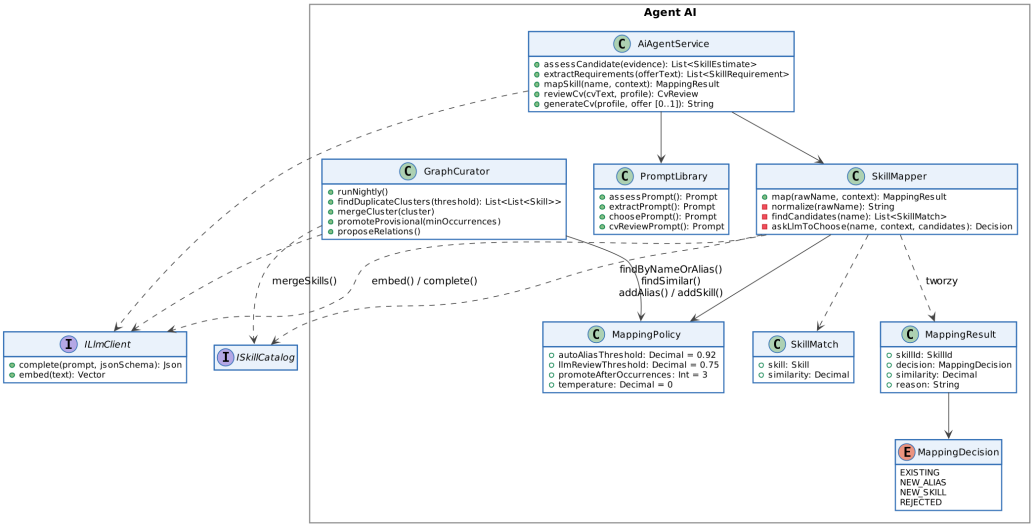


Diagram klas — oferty pracy i agent ofert

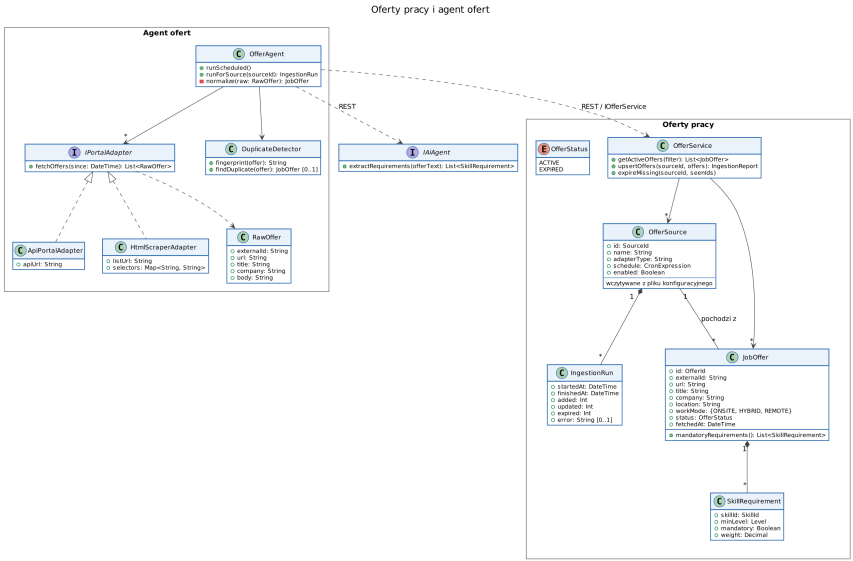
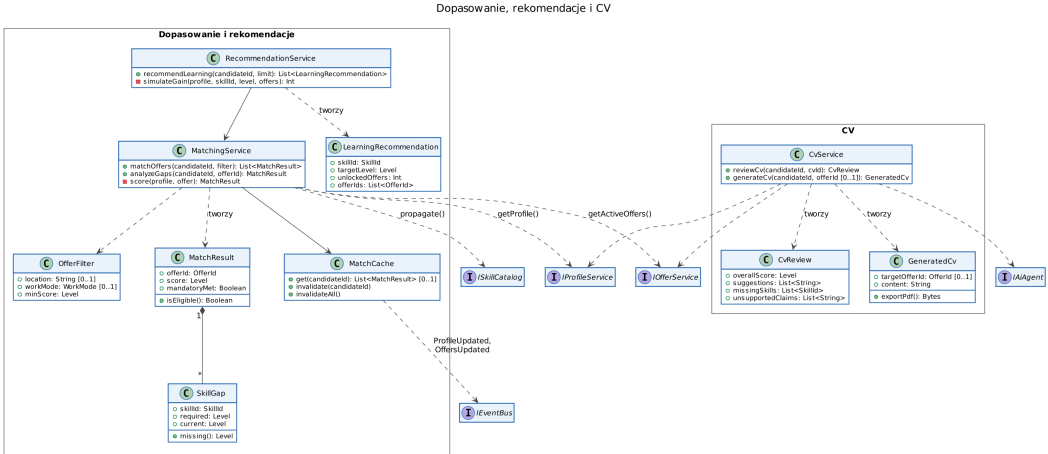


Diagram klas — dopasowanie, rekomendacje, CV



Otwarte pytania

- Z których portali pobieramy oferty i czy ich regulamin na to pozwala (API vs scraping)?
- Jak sprawdzać jakość ocen AI — ile ręcznych korekt to sygnał, że model się myli?
- Z jakiej klasyfikacji budujemy graf startowy (ESCO, tagi Stack Overflow, własna)?
- Jaki próg dopasowania uznajemy za „pasuje”?

Dziękujemy!